D - Computer Vision itu mudah bukan? Bukan

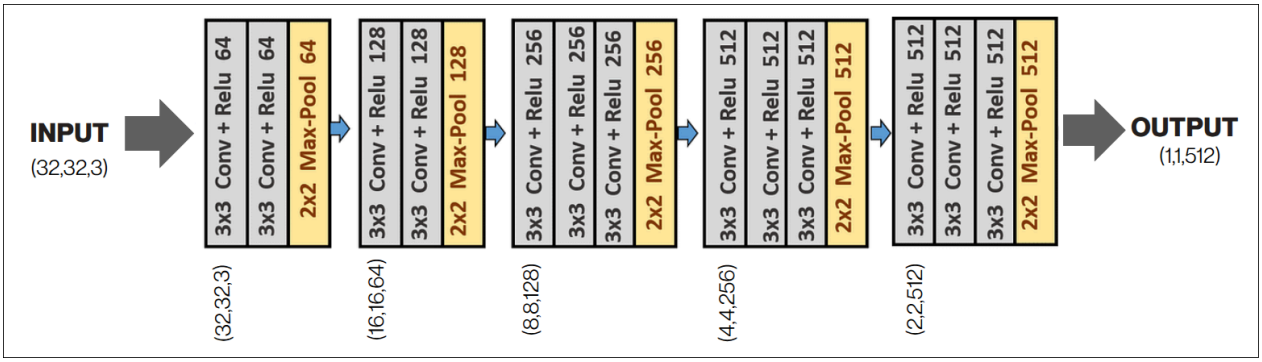
Author: RyZ
Time Limit: 10s
Memory Limit: 256 MB

Deskripsi Soal

Mas Ryu sangat suka dengan model Computer Vision. Saat ini, ia sedang mempelajari implementasi arsitektur Convolutional Neural Network dari awal. Salah satu arsitektur yang ingin diimplementasikan adalah **VGG16**.

VGG16 adalah salah satu model revolusioner yang dibuat oleh Visual Geometry Group dari Universitas Oxford pada tahun 2014. Idenya sebenarnya simpel, yaitu untuk melakukan ekstraksi gambar hingga ke bentuk yang sangat terkompresi sehingga, informasi didalamnya menjadi beharga dan mudah diinterpretasikan.

Arsitektur yang harus diimplementasikan terdiri atas lima blok berikut.



Setiap lapisan konvolusi menggunakan ketentuan berikut:

- ukuran kernel 3×3;
- `stride = 1`;
- `padding = 1`;
- bias bernilai nol;
- fungsi aktivasi ReLU; dan
- weight dibuat menggunakan Kaiming He Initialization.

Setiap operasi **max pooling** menggunakan:

- ukuran kernel 2×2; dan
- `stride = 2`.

Nilai `seed` digunakan agar bobot yang dihasilkan bersifat deterministik. Tiga belas lapisan konvolusi menggunakan seed secara berurutan mulai dari `seed`. Tugas Anda adalah melengkapi fungsi `vgg16` menggunakan fungsi-fungsi yang telah disediakan pada kode awal.

Input

Baris pertama berisi sebuah bilangan bulat S yang menyatakan nilai awal `seed`.

Sebanyak 3 baris berikutnya akan berisi nilai setiap pixel pada channel Red, Green, dan Blue.

```
R[0] R[1] R[2] ... R[1024]
G[0] G[1] R[2] ... G[1024]
B[0] B[1] B[2] ... B[1024]
```

Seluruh nilai piksel harus disimpan sebagai data bertipe `np.float32`.

Output

Cetak bentuk dari **feature map** yang dihasilkan setelah seluruh blok VGG16 selesai dijalankan dalam format:

`height width channels`

Pada baris berikutnya, cetak seluruh elemen dari **feature map** akhir yang telah diubah menjadi array satu dimensi menggunakan fungsi `flatten()`.

Setiap elemen dipisahkan oleh satu spasi dan ditampilkan dengan enam angka di belakang titik desimal.

Karena bentuk keluaran akhir adalah $1 \times 1 \times 512$, keluaran terdiri atas 512 bilangan riil.

```
1 1 512
value_1 value_2 value_3 ... value_512
```

Jawaban dianggap benar apabila memiliki kesalahan absolut atau relatif tidak lebih dari 10^{-5}.

Contoh

Input

```
https://pastebin.com/4Ahydrk2
```

Output

```
https://pastebin.com/Kr4NxqV2
```

Penjelasan

Gambar berukuran $32 \times 32 \times 3$ diproses oleh lima blok konvolusi VGG16.

Lapisan konvolusi menggunakan `padding = 1`, sehingga ukuran tinggi dan lebar **feature map** tidak berubah selama operasi konvolusi. Jumlah kanal berubah sesuai dengan jumlah **output channel** pada masing-masing lapisan.

Setiap akhir blok menggunakan **max pooling** berukuran 2×2 dengan `stride = 2`. Oleh karena itu, ukuran spasial berubah sebagai berikut:

```
32 × 32
16 × 16
8 × 8
4 × 4
2 × 2
1 × 1
```

Setelah blok kelima, diperoleh **feature map** dengan bentuk $1 \times 1 \times 512$.

Batasan

- $0 \leq S \leq 10^9$.
- Gambar selalu berukuran 32×32 dengan 3 channel (RGB).
- Setiap nilai piksel merupakan bilangan riil.
- 0.0 <= pixel <= 1.0.
- Seluruh perhitungan harus menggunakan tipe data `np.float32`.
- Hanya diperbolehkan menggunakan NumPy.
- Tidak perlu mengimplementasikan **backpropagation**.

TL;DR

Membuat `vgg16` yang menggabungkan seluruh *convolution layer* dan *max pooling* sesuai dengan susunan VGG16.

Fungsi ini menerima dua parameter:

- `input_data`, yaitu gambar masukan berukuran $32 \times 32 \times 3$; dan
- `seed`, yaitu nilai awal yang digunakan untuk membuat *weight*.

Pada soal ini, VGG16 terdiri atas lima blok.

Blok 1

```
Input
32 × 32 × 3

Conv2D(64)
32 × 32 × 64

Conv2D(64)
32 × 32 × 64

MaxPooling
16 × 16 × 64
```

Blok 2

```
Conv2D(128)
16 × 16 × 128

Conv2D(128)
16 × 16 × 128

MaxPooling
8 × 8 × 128
```

Blok 3

```
Conv2D(256)
8 × 8 × 256

Conv2D(256)
8 × 8 × 256

Conv2D(256)
8 × 8 × 256

MaxPooling
4 × 4 × 256
```

Blok 4

```
Conv2D(512)
4 × 4 × 512

Conv2D(512)
4 × 4 × 512

Conv2D(512)
4 × 4 × 512
```

```
MaxPooling
2 × 2 × 512
```

Blok 5

```
Conv2D(512)
2 × 2 × 512

Conv2D(512)
2 × 2 × 512

Conv2D(512)
2 × 2 × 512

MaxPooling
1 × 1 × 512
```

Secara keseluruhan, VGG16 pada soal ini memiliki `13` *convolution layer*:

$$2 + 2 + 3 + 3 + 3 = 13$$

Masing-masing *convolution layer* menggunakan `seed` yang sama secara berurutan:

```
Convolution layer 1
Convolution layer 2
Convolution layer 3
...
Convolution layer 13
```

Penggunaan urutan *seed* tersebut membuat setiap layer memiliki *weight* yang berbeda, tetapi tetap menghasilkan nilai yang sama ketika program dijalankan kembali menggunakan *seed* yang sama.

Setelah seluruh blok selesai dijalankan, bentuk *feature map* akhir adalah:

$$1 \times 1 \times 512$$

Dengan demikian, *feature map* akhir menyimpan `512` nilai.

Selanjutnya, Fungsi `flatten()` digunakan untuk mengubah tensor multidimensi menjadi array satu dimensi.

Pada akhir proses VGG16, *feature map* memiliki bentuk:

$$(1, 1, 512)$$

Agar seluruh nilainya dapat dicetak secara berurutan, program menjalankan:

```
flattened_output = output.flatten()
```

Setelah `flatten()` dijalankan, bentuk data berubah menjadi:

$$(512,)$$

Operasi `flatten()` hanya mengubah bentuk data. Nilai yang tersimpan di dalamnya tidak dihitung ulang dan tidak diubah.

Perubahannya dapat digambarkan sebagai berikut:

```
Sebelum flatten : 1 × 1 × 512
Setelah flatten : 512
```

Hasil akhirnya terdiri atas `512` bilangan riil yang kemudian dicetak dengan enam angka di belakang titik desimal.

Boilerplate

Berikut adalah kode yang sudah dibuat oleh mas Ryu

```
import numpy as np


def he_init(shape, seed=None):
    """He initialization."""
    rng = np.random.default_rng(seed)
    fan_in = shape[1] * shape[2] * shape[3]
    standard_deviation = np.sqrt(2.0 / fan_in)
    return rng.normal(
        loc=0.0,
        scale=standard_deviation,
        size=shape
    ).astype(np.float32)


def conv2d(input_data, weights, bias, stride=1, padding=0):
    """Operasi convolution 2D."""
    if padding > 0:
        input_data = np.pad(
            input_data,
            ((padding, padding), (padding, padding), (0, 0)),
            mode="constant"
        )

    input_height, input_width, input_channels = input_data.shape
    output_channels, weight_input_channels, kernel_height, kernel_width = weights.shape

    if input_channels != weight_input_channels:
        raise ValueError(
            f"Input memiliki {input_channels} channel, "
            f"sedangkan weight membutuhkan {weight_input_channels} channel."
        )

    output_height = (input_height - kernel_height) // stride + 1
    output_width = (input_width - kernel_width) // stride + 1
    output = np.zeros(
        (output_height, output_width, output_channels),
        dtype=np.float32
    )

    for output_channel in range(output_channels):
        current_weight = weights[output_channel].transpose(1, 2, 0)
        for h in range(output_height):
            h_start = h * stride
            h_end = h_start + kernel_height
            for w in range(output_width):
                w_start = w * stride
                w_end = w_start + kernel_width
                region = input_data[h_start:h_end, w_start:w_end, :]
                output[h, w, output_channel] = (
                    np.sum(region * current_weight) + bias[output_channel]
                )

    return output
```

```python
def relu(input_data):
    """ReLU activation."""
    return np.maximum(input_data, 0.0)


def conv_layer(
    input_data,
    output_channels,
    kernel_size=3,
    stride=1,
    padding=1,
    seed=None
):
    """Satu convolutional layer: He init, Conv2D, ReLU."""
    input_channels = input_data.shape[2]
    weight_shape = (
        output_channels,
        input_channels,
        kernel_size,
        kernel_size
    )
    weights = he_init(shape=weight_shape, seed=seed)
    bias = np.zeros(shape=(output_channels,), dtype=np.float32)
    output = conv2d(
        input_data=input_data,
        weights=weights,
        bias=bias,
        stride=stride,
        padding=padding
    )
    return relu(output)


def max_pooling(input_data, pool_size=2, stride=2):
    """Operasi max pooling."""
    input_height, input_width, input_channels = input_data.shape
    output_height = (input_height - pool_size) // stride + 1
    output_width = (input_width - pool_size) // stride + 1
    output = np.zeros(
        (output_height, output_width, input_channels),
        dtype=np.float32
    )

    for channel in range(input_channels):
        for h in range(output_height):
            h_start = h * stride
            h_end = h_start + pool_size
            for w in range(output_width):
                w_start = w * stride
                w_end = w_start + pool_size
                region = input_data[
                    h_start:h_end,
                    w_start:w_end,
                    channel
                ]
                output[h, w, channel] = np.max(region)

    return output
```